

같은 피규어인데
매장마다 가격이 다르다.

STORE A

~~₩~~₩28,000

오프라인 매장 A

VS

STORE B

~~₩~~₩31,000

오프라인 매장 B

실상은, 정가

~~₩~~20,000

MSRP — 제조사 발매 정가

매장에서 4천 ~ 1만원씩 더 받는 피규어가 흔하다.

그래서 — 개발한 시스템.

01

사진 한 장

스마트폰으로 찍은 사진을 업로드

02

자동 식별

DINOv2 + 벡터 검색으로 12K DB 매칭

03

정가 · 시세

정가와 현재 파트너 호가를 한눈에

FigurePrice

사진 한 장 → 모델 인식 + 정가 · 시세 분포

프로젝트 소개

문제

- 피규어를 들고 있거나 사진만 봤는데 이름·제조사·가격을 모를 때
- 구글 이미지 검색은 같은 캐릭터의 다른 스컬프트·스케일 구분이 약함

한국에선 시세를 한눈에 보기 어려움 — YJP / eBay / MFC 등 분산

해결

- 사진 한 장 업로드 → 시각 임베딩으로 12,019개 figure DB와 매칭
- 정가 + 현재 파트너 스토어 호가 (JPY/USD → KRW 환산)

DB에 없으면 Gemini가 Google Search로 웹에서 식별 후 결과 표시

오프라인 + 로컬 전용 데스크톱 앱. 외부 서버 안 띄움.

BY THE NUMBERS

데이터셋 & 기술 스택

12,019

FIGURES

MFC 크롤 (1/7 · 1/8 외 다양)

12,028

EMBEDDINGS

DINOv2-Large · 1024-dim

20,149

PARTNER PRICES

실시간 파트너 호가

STACK

VISION

DINOv2-Large + rembg

MPS 가속 (Apple Silicon)

VECTOR

sqlite-vec (vec0)

L2 = 코사인 거리

LLM

Gemini 3.1-flash-lite /
2.5-flash

Google Search Grounding

UI

PySide6 + matplotlib

Spotify 다크 테마(DESIGN.md)

MODULE TREE

구현 구조

·	
├── gui/	PySide6 데스크톱 앱
│ ├── main_window.py	QMainWindow + 페이지 스택
│ ├── widgets/	UploadView · ResultView · CandidateCard
│ │ ├── PriceChart · GeminiLookupPanel · AddToDb	
│ ├── workers.py	QThread (임베딩 · Gemini · 크롤)
│ └── theme.py	Spotify 다크 QSS
├── services/	GUI가 직접 호출하는 순수 함수
│ ├── identify.py	사진 → 임베딩 → 벡터 검색 → rerank
│ ├── lookup.py	Gemini Google Search 그라운드링
│ ├── mutations.py	confirm_image / add_manual / add_by_mfc
│ └── types.py	FigureCandidate · PriceStats · ...
├── embeddings/embedder.py	DINOv2-large + 5-crop TTA
├── crawler/mfc_crawler.py	MFC 크롤러 (메타 + 이미지 + 호가)
├── db/	SQLite + sqlite-vec (vec0)
└── data/figures.db	12K figures · 200MB images

R U N

python -m gui

(또는 FigurePrice.command 더블클릭)

A S Y N C

Worker는 QThread로 분리.

UI가 절대 얼지 않음.

N O H T T P

Service layer는 HTTP 없는

순수 파이썬 함수.

B A C K E N D O N L Y

FastAPI · React 전부 제거.

실행 파일은 결국 Python 하나.

RUNTIME PIPELINE

프로그램 실행 흐름

- 1

사용자 사진 업로드
- 2

rembg: 배경 제거 → 흰 배경 합성
- 3

Multi-crop fanout (원본 + 4 변형)
- 4

DINOv2-large CLS 토큰 × 5 (MPS)
- 5

sqlite-vec MATCH × 5 (top-30 each)
- 6

figure_id 별 min-distance 머지
- 7

선택: Gemini 재랭크 (top-K + 쿼리)
- 8

Best 카드 + 정가 + 파트너 호가 차트

FALLBACK

매칭 신뢰도가 낮을 때

- Gemini + Google Search
- 후보 3개 (이미지 + 메타 + 호가)

사용자가 힌트 텍스트로 재검색

응답 시간

2~3초

첫 호출 시 모델 로드 ~45초

CONFIDENCE

HIGH

MID

LOW

LIVE WALKTHROUGH

데모



[영상 자리]

* 실제 시연 영상입니다.

업로드 → DINOv2 매칭 → 가격 차트 → Gemini 폴백 → DB 추가 → 사진 확인 학습

회고 & 다음 단계

AI 받은 부분 vs 직접 한 부분

AI 도움

- 초기 스캐폴딩 (FastAPI → 이후 폐기)
- PySide6 위젯 보일러플레이트

디자인 시스템 → Qt QSS 변환

직접 작업

- MFC HTML 셀렉터 실측 디버깅
- Multi-crop test-time augmentation
- Gemini API 연결 (3.1 / 2.5)

DB 스키마 마이그레이션 (UNIQUE 제거)

한국어 폰트 매칭 (matplotlib)

어려웠던 점

ISSUE

- MFC 크롤 503/403 (rate limit) — 1초 폴라이트 페이스로 조정
- Gemini 한국 계정 무료티어 모델별 한도 다름
- 클로즈업 사진 정확도 — 풀바디 레퍼런스와 도메인 갭

PySide6 QThread worker GC 이슈

Service layer 추출 시 의존성 정리

시간이 더 있다면

NEXT

- 레퍼런스 multi-crop 인덱싱 (60K 벡터)
—> 클로즈업 정확도 ↑
- CLIP 보조 점수 (DINOv2 + CLIP 하이브리드)
- Yahoo Auctions JP / Mercari 낙찰가 추가

가격 변화 추적 그래프 (시계열 partner_prices)

모바일 앱 (iOS Catalyst)